Individual Final Report: Smart Resume Builder

Name: Dinesh Chandra Gaddam

GitHub Repository: https://github.com/imchandanmohan/smart-resume-builder

---

Content

---

1. Introduction

The Smart Resume Builder is a web-based application designed to help users create professional resumes efficiently. It integrates Natural Language Processing (NLP), Machine Learning (ML), and Large Language Models (LLMs) to deliver personalized resume suggestions, optimize keywords, and ensure Applicant Tracking System (ATS) compliance.

In this collaborative project, our team contributed as follows:

- Frontend Development: Built an interactive Streamlit user interface to manage file uploads, display processing progress, and preview the final PDF.
- Backend Development: Implemented job description extraction (Draft 1) using MistralAI to structure and process resumes against job requirements.
- NLP & ML Components:
    - Parsed job descriptions to extract roles, responsibilities, and required skills.
    - Performed keyword extraction and semantic matching using transformer-based models.
- ATS Interlinking: Encoded ATS rules, regulations, and formatting constraints into prompt templates to guide the LLM in generating ATS-compliant sections.
- Mailing User his resume after building the user resume.

---

2. Description of My Individual Work

This section details my individual contributions aligned with each major component outlined in the Introduction. Background theory, algorithmic insights, essential equations, and figure placeholders are provided for each.

2.1 Frontend Development

Background: Leveraged Streamlit's declarative, widget-driven framework to construct an intuitive UI. I designed a finite-state machine to manage page flow (upload, analysis, final), ensuring persistence of uploaded files and generated content across navigation events.

Algorithm:

- Initialize st.session_state.page to track the current view.
- Define button callbacks to transition between states and rerun the script declaratively.

Figure 1: Frontend State Machine Diagram (Depicts UI states and transitions.)

2.2 Backend Development (Job Description Extraction)- Initial Draft

Background: Architected a modular pipeline using MistralAI via Together API to extract structured data from raw job descriptions. The stages include text normalization, transformer-based NER, QA span extraction, and embedding-based keyword ranking.

Transformation Function:

Output_s = LLM(R_s, D_s, C_s)

- R_s: Section-specific rules loaded from prompt_data.py.
- D_s: Excerpt of the job description relevant to sections.
- C_s: Candidate's original resume content for sections.

2.3 NLP & ML Components- Initial Draft

Background: Focused on five key sections—Contact, Skills, Technical Experience, Professional Experience, and Education—by combining prompt rules with transformer-based models for content matching and formatting.

- Contact Parsing: Employed mistral to extract location, email, phone, and URLs. Prompts enforce a single-line format and pipe-separated fields per prompt_data.py rules.
- Skills Extraction: Extracted relevant skills using mistral

- Technical Experience: Parsed project entries into name, description, technologies, and outcomes. Prompt rules specify grouping of technologies and concise outcome statements.
- Professional Experience: Used a QA pipeline to extract responsibilities, then reformatted them into action-verb-led bullet points per style guidelines.
- Education Details: Extracted degree, institution, graduation date, and awards. Prompt rules enforce reverse-chronological order and concise award grouping.

2.4 LaTeX Integration- Initially planned for the final page, Not implemented in current Presentation

Background: Created a dynamic LaTeX template (templete.tex) with named placeholders for each section. A Python script reads the template, replaces tokens with generated content, and compiles the PDF via pdflatex**.**

2.5 ATS Interlinking- Initial Draft

Background: Integrated ATS rules, regulations, and formatting constraints from prompt_data.py into the LLM prompt templates for Contact, Skills, Technical Experience, Professional Experience, and Education. This ensured each generated section adhered to ATS best practices without an explicit scoring mechanism**.**

2.6 Group Proposal:

The Proposal was built to initial expectations of the project. Not to the adjusted to the time constraint changes made and committed the Initial proposal. In the proposal the questions asked were answered.

Detailed Description of My Contributions

3.1. UI & Navigation Flow

- Objective: Provide a seamless, multi-step interface for users to upload files, view processing status, and preview/download the final resume. UI is based on JobScan page
- Implementation Details:
    - Designed three main views—Upload, Analysis, Final—managed via Streamlit session state.
    - Employed responsive layout components (columns, buttons) to guide users through each stage.
    - Displayed live progress indicators (spinners, status messages) during backend operations.
    - Maintains memory throughout the session. Once refreshed or restarted then new resume and Job description should be added.

3.1.1.

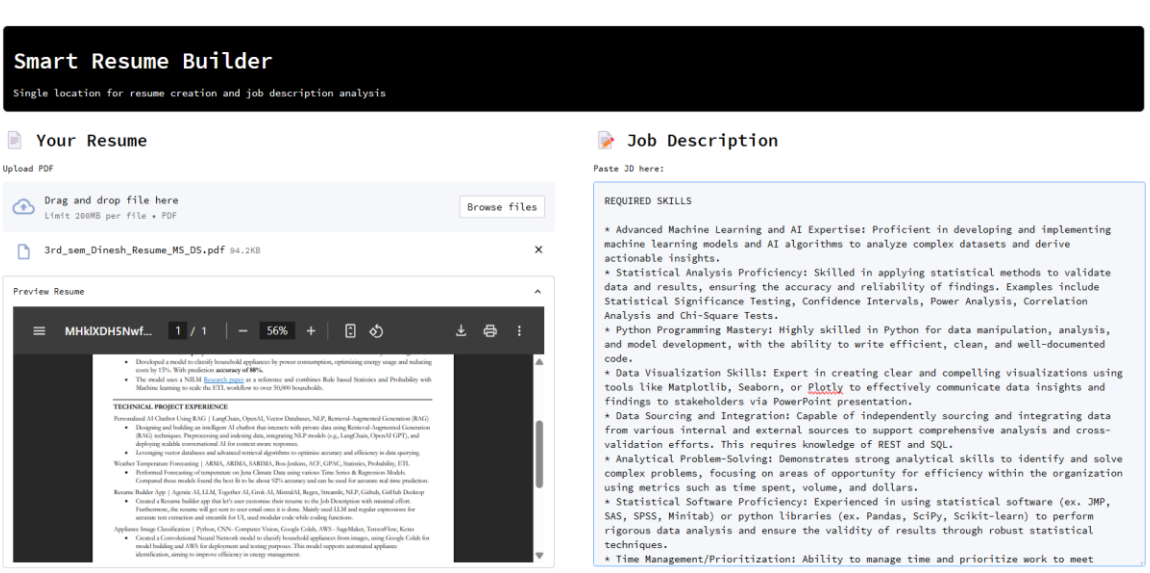

Fig-1

- This is the First page of the UI involves user uploading PDF, pasting Job Description, Header, Analyse button that links this page to the next page.

3.1.2

This page is a vital one where all the analysis happens

- The Side bar has ATS score of the uploaded Resume w.r.t Job description and few optimisations that can be done to the resume. Issues to Fix is what I called it. View in Fig-2
- Header is common for all the pages
- Mainbar has all the mistakes made in the resume which will be fixed in the next step
- This page also includes three different sub headers as it is visible in UI. They are ATS analytics which is the landing page, next to it we have Job Description and Resume which have annotated highlighting keywords, verbs and entity recognition
- At the bottom of the page we have back option and next option

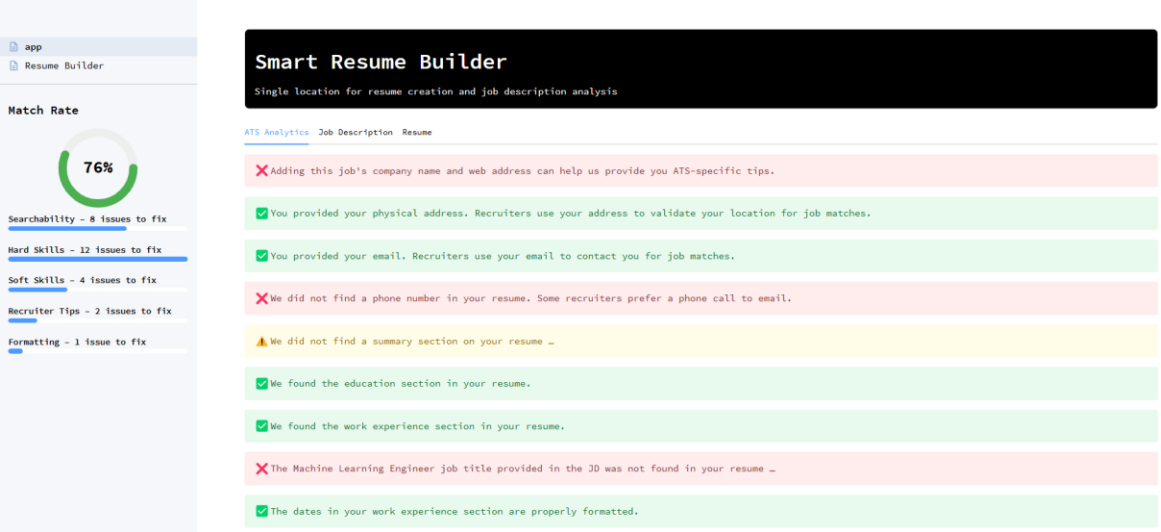

Fig-2

3.2. Job Description Parsing Module- Initial Draft

- Objective: Extract structured fields (job title, company, location, responsibilities, skills, qualifications, experience, keywords, benefits, job type) from raw job descriptions using a prompt-based LLM approach.
- Implementation Details:
  - Crafted targeted prompts for each field to guide the LLM.
  - Post-processed responses by splitting lines and stripping list markers.
  - Applied regex fallbacks to handle missing or malformed outputs.
- Pre-processing: Passed raw text directly to the LLM without manual cleaning; relied on prompt instructions for normalization.

3.3 Resume Content Modification-Initial Draft

- Contact Section Processing
    - Objective: Normalize and format user contact information (location, LinkedIn URL, phone, email) into a single, ATS-friendly line.
    - Implementation Details:
        - Used named-entity recognition to identify location fields.
        - Applied pattern matching to validate emails and phone numbers.
        - Enforced pipe-separated formatting according to ATS guidelines.
    - Figure: *Contact Extraction & Formatting Pipeline*
- Skills Extraction & Ranking
    - Objective: Identify and prioritize the most relevant skills from both job description and candidate profile.
    - Implementation Details:
        - Computed term weights to filter out low-value tokens.
        - Generated semantic embeddings to measure similarity between job and candidate skills.
        - Ranked skills by relevance and trimmed to the top 5–10 entries.
- Technical Experience Parsing
    - Objective: Decompose project entries into structured elements—project name, description, technologies used, and key outcomes.
    - Implementation Details:
        - Unified list vs. string inputs to ensure consistent prompt inputs.
        - Guided the LLM to group technologies distinctly and craft concise outcome statements.
    - Figure: *Technical Projects Prompt Flow*
- Professional Experience Enhancement
    - Objective: Rewrite work-experience bullet points to mirror job description language, emphasize metrics, and use strong action verbs.
    - Implementation Details:
        - Mapped extracted responsibilities to job requirements for semantic alignment.
        - Cleaned and formatted output to match bullet-point style guidelines.

        - Logged and retried any failed API calls to ensure robustness.
    - Figure: *Work Experience Transformation Pipeline*
- Education Section Formatting
    - Objective: Enforce reverse-chronological ordering of degrees and concise grouping of awards or honors.
    - Implementation Details:
        - Parsed and converted graduation dates for proper sorting.
        - Combined multiple awards into a single line when applicable.
    - Figure: *Education Formatting Diagram*
- Pre-processing & Session Management
    - Objective: Ensure consistent data handling and persistent user state across the Streamlit UI.
    - Implementation Details:
        - Unified data structures (lists vs. strings) before prompting.
        - Utilized session state to store uploads, extracted content, and generated sections throughout user navigation**.**

3.4. LaTeX Template Integration- Currently Not Implemeted

- Objective: Apply a professional, consistent layout to the generated resume content using a pre-designed .tex template.
- Implementation Details:
    - Built templete.tex with named placeholders for each section (e.g., {{Name}}, {{WorkExperience}}).
    - Ensured styling rules—fonts, margins, section headers—matched industry standards.
    - Automated placeholder replacement in Python before invoking the LaTeX compiler.

3.4.1 PDF Rendering & Download

- Objective: Compile the filled LaTeX template into a PDF and present it directly in the web app for preview and download.
- Implementation Details:
    - Launched a subprocess call to pdflatex in a temporary directory.
    - Captured compiler logs to detect and report errors gracefully.

- Embedded the resulting PDF in the Final view via Streamlit's file-display widget and provided a download button.□ ATS Rules Interlinking

- Objective: Enforce ATS-compliance by incorporating formatting and keyword guidelines into every generation prompt.
- Implementation Details:
  - Loaded section-specific ATS rules from prompt_data.py (e.g., header names, bullet counts, keyword thresholds).
  - Injected these rules at the top of each prompt to the LLM, ensuring outputs adhered to formatting constraints without a separate scoring pass.

3.4.2. Application Orchestration & Error Handling

- Objective: Coordinate frontend interactions with backend processes and maintain robustness under failure conditions.
- Implementation Details:
  - Centralized orchestrator function to sequence parsing, transformation, cleaning, and PDF generation steps.
  - Wrapped all external calls (LLM, file I/O, subprocess) in try/except blocks, logging issues and providing user-friendly error messages.

4. Results

4.1 Resume Scoring Performance

Resume	Job Description	Match Score (%)	Analysis- Missing skills, optimizations
Resume1	JobDesc1	85%	Searchability-8, Hard and Soft Skills-2 etc
Resume2	JobDesc2	72%	Searchability-4, Hard and Soft Skills-6 Fig-2

Interpretation:

- Higher scores indicate better alignment with job requirements.
- Missing keywords help users improve their resumes. Fig-2 tells us about this

4.2 **Skill Gap View**

- In the Analysis Page or Page-2 of UI we can see the skill gap results of resume vs Job description..

---

5. Summary & Conclusions

Key learnings

- I have learned to Use Streamlit and was create UI and customise to my needs.
- Learned to use Latex and implemented it to create the final Resume.
- Created Draft-1 of Job description extraction involving learning about LLMs and worked with MistralAI.
- Learned to work with complex integration of different modules importing them and using LLM as API references
- Learned to connect and use Gmail to send an automated email.
- Finally learned to implement Agentic Workflow with memory, Automated task with few clicks and successfully implemented Resume Builder.ai

Future Improvements

- Adding Voice prompting so that user can add extra information that are not added in resume.
- The page below is the final page of the App. Has the user's Resume updated to Job description.
- User can Download the resume or can email it to himself. The email itself is extracted from the resume the user uploaded.

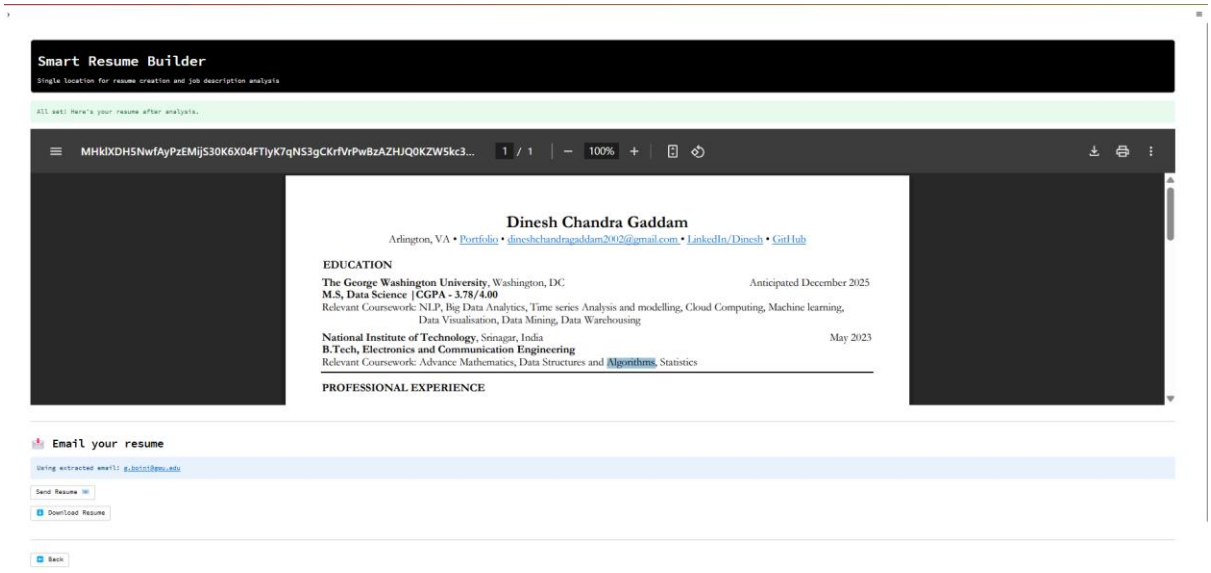

Fig-3

6. Percentage of Code from Internet

- Total Lines of Code (My Contribution): 900
- Code I found in internet like streamlit and GPT etc: 600
- Modified Code: 400 lines with according to architecture, played with UI and other files for best experience.
- Own Code: 300 ~ approx making it modular and accessible and rules and hand typed.

Percentage=600-400/600+300=20%

Conclusion: 20% of the code was adapted from external sources. (Available at Contributions)

---

7. References

1. Streamlit Documentation and playground: streamlit
2. Scikit-learn TF-IDF. https://scikit-learn.org/
3. NLTK Text Processing. https://www.nltk.org/
4. Together AI Docs**:** together ai
5. Jobscan web page: UI reference

Note:

The Implementation of UI, Job Description Extraction is directly implemented in the final Demo with little modification and rest of the files are the Drafts that helped my team mates and The Estimated Final Page of modified Resume for User. Please review the Contribution and Commits for More Information.